МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интелекта
02.03.01 Математика и компьютерные науки

Отчет по лабораторной работе №1

Дискретная математика
«Операции над мультимножествами и код Грея»

Выполнил:
студент группы 5130201/50301 _____ Ташлыкова Е.В.
Принял:
Преподаватель _____ Востров А.В.

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение 4

1 Математическое описание 5

1.1 Множества и мультимножества 5

1.2 Операции над мультимножествами 6

1.3 Алгоритм генерации бинарного кода Грея 8

2 Особенности реализации 10

2.1 Структуры данных 10

2.2 Описание функций 11

2.3 Алгоритм автоматического заполнения 28

2.4 Ручное заполнение мультимножеств 29

2.5 Защита от некорректного ввода 30

3 Результаты работы 32

3.1 Главное меню 32

3.2 Создание универсума 32

3.3 Автоматическое заполнение 32

3.4 Ручное заполнение 33

3.5 Выполнение операций 33

3.6 Граничные случаи 33

3.7 Время выполнения этапов 33

3.8 Некорректный пункт меню 34

3.9 Некорректная разрядность 34

3.10 Мощность больше мощности универсума 34

3.11 Ошибки при ручном вводе 35

3.12 Операции над пустыми мультимножествами 35

Заключение 36
Список литературы 38
Приложение. Исходный код программы 39

Введение

Цель работы – получение знаний о принципе работы с мультимножествами, алгоритме генерации бинарного кода Грея, а также реализация данных алгоритмов в виде программы на языке программирования C++.

Задание: реализовать программу генерации бинарного кода Грея для заполнения универсума мультмножеств (заданной пользователем разрядности). На основе универсума формируются два мультимножества двумя способами заполнения – вручную и автоматически (выбирает пользователь). Мощности множеств задает пользователь. В результате на экран выводятся результаты действий над множествами: объединение, пересечение, разность (оба варианта), симметрическая разность, дополнение (оба варианта), арифметические сумма, разность, произведение и деление. Кроме того, требуется реализовать защиту от некорректного пользовательского ввода и пользовательское меню.

1 Математическое описание

1.1 Множества и мультимножества

Понятие множества принадлежит к числу фундаментальных понятий математики. Множество – это любая определенная совокупность объектов. Объекты, из которых составлено множество, называются его элементами, при этом все элементы множества различны. (Новиков, стр 23).

Если объект x является элементом множества M, то говорят, что x принадлежит M ($x \in M$). В противном случае говорят, что x не принадлежит M ($x \notin M$). Множество, не содержащее элементов, называется пустым и обозначается как $\varnothing$.

Мультимножеством называется множество, в котором допускается наличие одинаковых элементов. Пусть дано множество $X = x_1, \ldots, x_n$, тогда мулитмножеством над множеством X будет являться $\widehat{X} = [x_1^{a_1}, \ldots, x_n^{a_n}] =$

$= \langle x_1, \ldots, x_1; \ldots; x_n, \ldots, x_n \rangle = \langle a_1(x_1), \ldots, a_n(x_n) \rangle$. Это совокупность элементов множества X, в которую элемент x_i входит $a_i \geqslant 0$ раз.

Число a_i называется *показателем* элемента x_i, множество X – *носителем*, число $m = a_1 + \cdots + a_n$ – *мощностью*, а $\underline{\widehat{X}} = \{x_i \in X \mid a_i > 0\}$ называется *составом* мультимножества $\widehat{X}$.

Элемент x принадлежит мультимножеству $\widehat{X}$ ($x \in \widehat{X}$) и в мультимножестве $\widehat{X}$ имеется ровно k экземпляров элемента x тогда и только тогда, когда кратность элемента x равна $k_X(x) = k > 0$. Когда кратность элемента x равна нулю $k_X(x) = 0$, тогда элемент x не содержится в мультимножестве $\widehat{X}$ ($x \notin \widehat{X}$).

Принадлежность элемента x мультимножеству $\widehat{X}$ определяется функцией

кратности:

$$k_X(x) = \begin{cases} k > 0 \text{ – целое, если } x \in X, \\ 0, \text{ если } x \notin X. \end{cases}$$

В работе универсумом U называется мультимножество над множеством X кодов Грея, содержащее все допустимые элементы с максимальными кратностями. Мультимножество A является подмультимножеством универсума ($A \subseteq U$), если $m_A(x) \leqslant m_U(x)$ для всех $x \in X$.

Элементами множества X являются коды Грея разрядности n, а их кратности в U устанавливаются случайным образом из диапазона [0, 7].

1.2 Операции над мультимножествами

- Объединением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые присутствуют хотя бы в одном из мультимножеств, и кратность каждого элемента равна максимальной кратности соответствующих элементов в объединяемых мультимножествах

$$A \cup B = \{\, k_{A \cup B}(x) \mid k_{A \cup B}(x) = \max\{k_A(x),\, k_B(x)\} \,\}$$

- Пересечением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна минимальной кратности соответствующих элементов в пересекаемых мультимножествах

$$A \cap B = \{\, k_{A \cap B}(x) \mid k_{A \cap B}(x) = \min\{k_A(x),\, k_B(x)\} \,\}$$

- Дополнением мультимножества A до универсума U называется мультимножество, состоящее из всех элементов универсума и кратность каждого

элемента равна разности кратностей соответствующих элементов в универсуме U и дополняемом мультимножестве A

$$\bar{A} = \{\, k_{\bar{A}}(x) \mid k_{\bar{A}}(x) = k_U(x) - k_A(x) \,\}$$

- Разностью (теоретико-множественной) называется мультимножество, состоящее из минимума кратностей первого мультимножества и разности кратностей универсума и второго мультимножества

$$A \setminus B = \{\, k_{A\setminus B}(x) \mid k_{A\setminus B}(x) = \min\{k_A(x),\, k_U(x) - k_B(x)\} \,\}$$

- Симметрической разностью мультимножеств A и B называется мультимножество, состоящее из всех элементов обоих мультимножеств, кратности которых равняются объединению кратностей двух разностей мультимножеств – из первого второго и из второго первого

$$A \triangle B = \{\, k_{A\triangle B}(x) \mid k_{A\triangle B}(x) =$$

$$= \max\{\min\{k_A(x),\, k_U(x) - k_B(x)\},\, \min\{k_B(x),\, k_U(x) - k_A(x)\}\} \,\} =$$

$$= (A \cup B) \setminus (A \cap B)$$

- Арифметической суммой мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые присутствуют хотя бы в одном из мультимножеств, и кратность каждого элемента равна сумме кратностей соответствующих элементов в складываемых мультимножествах

$$A + B = \{\, k_{A+B}(x) \mid k_{A+B}(x) = \min\{k_A(x) + k_B(x),\, k_U(x)\} \,\}$$

- Арифметической разностью мультимножеств A и B называется мультимножество, состоящее из всех элементов мультимножества A, кратность

каждого элемента равна разности кратностей соответствующих элементов в вычитаемых мультимножествах

$$A - B = \{\, k_{A-B}(x) \mid k_{A-B}(x) = \max\{k_A(x) - k_B(x),\, 0\} \,\}$$

- Арифметическим произведением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна произведению кратностей соответствующих элементов в перемножаемых мультимножествах

$$A * B = \{\, k_{A*B}(x) \mid k_{A*B}(x) = \min\{k_A(x) * k_B(x),\, k_U(x)\} \,\}$$

- Арифметическим делением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна целочисленному делению кратностей соответствующих элементов, при условии, что кратность элемента в делителе не равна нулю

$$A : B = \begin{cases} \{\, k_{A:B}(x) \mid k_{A:B}(x) = [k_A(x) : k_B(x)] \,\}, & k_B(x) \neq 0 \\ 0, & k_B(x) = 0 \end{cases}$$

1.3 Алгоритм генерации бинарного кода Грея

n-Мерный код Грея – это последовательность всех 2^n бинарных слов длины n такая, что два соседних слова отличаются ровно в одном символе. Известно несколько алгоритмов построения кодов Грея. Традиционно используют нерекурсивный алгоритм, основанный на операции побитового исключающего ИЛИ $\oplus$ исходного двоичного кода N-разрядного числа и его копии сдвинутой на один

разряд вправо.

$$G = BIN \oplus (BIN \gg 1) \tag{1}$$

где $\gg$ – операция сдвига вправо на один разряд.

Далее, в таблице 1, приведен пример для $n = 3$.

Таблица 1 – Двоичный код Грея для $n = 3$

Десятичное число	BIN	$BIN \gg 1$	Код Грея (G)
0	000	000	000
1	001	000	001
2	010	001	011
3	011	001	010
4	100	010	110
5	101	010	111
6	110	011	101
7	111	011	100

2 Особенности реализации

2.1 Структуры данных

Для хранения данных о мультимножествах и управления ими (включая универсальное) реализован класс `MultiSet`.

Листинг 1: Класс мультимножества `MultiSet`

```
class MultiSet
{
public:
    using Count = std::size_t;
    using Data = std::map<std::string, Count>;
    MultiSet() = default;
    void clear();
    void add(const std::string &value, Count count = 1);
    Count count(const std::string &value) const;
    Count size() const;
    bool empty() const;
    const Data &data() const;
private:
    Data data_;
};
```

С помощью псевдонимов `Count` и `Data` определяются типы: `Count` (синоним `std::size_t`) задает тип для хранения кратности элементов, а `Data` (синоним `std::map<std::string, Count>`) – тип внутренней структуры данных, представляющей собой упорядоченный словарь, где ключом выступает строковое значение элемента кода Грея, а значением – его кратность в мультимножестве.

Конструктор по умолчанию `MultiSet()` создает пустое мультимножество.

Метод `clear()` полностью очищает контейнер, удаляя все элементы.

Метод `add(const std::string &value, Count count = 1)` добавляет указанный элемент в мультимножество, увеличивая его текущую кратность на заданное значение (по умолчанию на единицу).

Метод `count(const std::string &value)` возвращает текущую кратность конкретного элемента (возвращает 0, если элемент отсутствует).

Метод `size()` вычисляет и возвращает общую мощность мультимножества, то есть сумму кратностей всех его уникальных элементов.

Метод `empty()` проверяет, является ли мультимножество пустым.

Метод `data()` возвращает константную ссылку на внутренний словарь, что позволяет обходить все элементы и получать их кратности без возможности модификации внутреннего состояния объекта.

2.2 Описание функций

2.2.1 decimalToBinary

Вход: value (unsigned long) – число для перевода; width (std::size_t) – длина результирующей строки.

Выход: строка (std::string) длиной width, содержащая двоичное представление числа.

Описание: функция создаёт строку из width нулей и заполняет её битами числа value, начиная со старшего разряда. Для каждой позиции выполняется сдвиг числа вправо на нужное количество разрядов и проверка младшего бита. Если число содержит меньше разрядов, чем width, строка дополняется ведущими нулями.

Листинг 2: Функция `decimalToBinary`

```
std::string decimalToBinary(unsigned long value, std::size_t width){
    std::string result(width, '0');
    for (std::size_t i = 0; i < width; ++i){
        const std::size_t shift = width - 1 - i;
        result[i] = ((value >> shift) & 1ULL) ? '1' : '0';
    }
    return result;
}
```

2.2.2 generateGrayCode

Вход: bits (std::size_t) – разрядность кода.

Выход: вектор строк (std::vector<std::string>) – последовательность кодов Грея длиной bits символов.

Описание: функция генерирует все 2^{bits} кодов Грея заданной разрядности. Для каждого номера i от 0 до $2^{bits} - 1$ вычисляется значение $i \oplus (i \gg 1)$, которое затем переводится в двоичную строку фиксированной длины с помощью функции `decimalToBinary`. При нулевой разрядности возвращается пустой вектор.

Листинг 3: Функция `generateGrayCode`

```
std::vector<std::string> generateGrayCode(std::size_t bits){
    std::vector<std::string> result;
    if (bits == 0) return result;
    const std::size_t count = 1 << bits;
    result.reserve(count);
    for (std::size_t i = 0; i < count; ++i){
        unsigned long gray = i ^ (i >> 1);
        result.push_back(decimalToBinary(gray, bits));
    }
    return result;
}
```

2.2.3 readInt

Вход: prompt (const std::string&) – приглашение к вводу; minValue (int) – нижняя граница допустимых значений; maxValue (int) – верхняя граница допустимых значений.

Выход: введённое пользователем целое число (int) из диапазона [minValue, maxValue].

Описание: функция в цикле выводит приглашение и считывает строку целиком. Строка разбирается как одно целое число без посторонних символов, после чего проверяется принадлежность числа заданному диапазону. При некорректном вводе выводится сообщение об ошибке и запрос повторяется, пока не будет введено допустимое значение.

Листинг 4: Функция `readInt`

```
int readInt(const std::string &prompt, int minValue, int maxValue){
    while (true){
        std::cout << prompt;
        std::string line;
        std::getline(std::cin, line);
        std::stringstream ss(line);
        long value;
        char extra;
        if ((ss >> value) && !(ss >> extra) && value >= minValue && value <=
   maxValue)
            return static_cast<int>(value);
        std::cout << "Ошибка: введите целое число от " << minValue << " до "
   << maxValue << ".\n";
    }
}
```

2.2.4 readManualMultiset

Вход: name (const std::string&) – имя заполняемого мультимножества; universe (const MultiSet&) – универсум; order (const std::vector<std::string>&) – порядок элементов универсума для нумерации; power (std::size_t) – требуемая мощность мультимножества.

Выход: заполненное мультимножество (MultiSet) мощности power.

Описание: функция реализует ручное заполнение мультимножества. Пользователь power раз вводит номер элемента универсума, и соответствующий элемент добавляется в результат. Если кратность выбранного элемента в результате уже достигла его кратности в универсуме, выводится сообщение об ошибке, а ввод для этого вхождения повторяется. Если power равно нулю или универсум пуст, возвращается пустое мультимножество.

Листинг 5: Функция `readManualMultiset`

```
MultiSet readManualMultiset(const std::string &name, const MultiSet &universe,
                            const std::vector<std::string> &order, std::size_t power)
{
    MultiSet result;
    if (power == 0 || universe.empty()) return result;
    std::cout << "\nРучное заполнение мультимножества " << name << ".\n";
    std::cout << "Введите номер элемента универсума для каждого вхождения.\n";
    for (std::size_t i = 0; i < power; ++i){
        const std::size_t index = readInt(
            "Элемент " + std::to_string(i + 1) + ": ",
            1, order.size());
        const std::string &chosen = order[index - 1];
        if (result.count(chosen) >= universe.count(chosen)){
            std::cout << "Ошибка: элемент " << chosen
                      << " уже использован максимальное число раз ("
```

```
                        << universe.count(chosen) << ").\n";
            --i;
            continue;
        }
        result.add(chosen);
    }
    return result;
}
```

2.2.5 generateAutomaticMultiset

Вход: universe (const MultiSet&) – универсум; power (std::size_t) – требуемая мощность мультимножества.

Выход: случайно сформированное мультимножество (MultiSet), являющееся подмультимножеством универсума.

Описание: функция строит список, в котором каждый элемент универсума повторяется столько раз, какова его кратность. Затем список перемешивается генератором псевдослучайных чисел, и из него берутся первые power элементов, которые добавляются в результат. Если power превышает мощность универсума, берутся все элементы. При нулевой мощности или пустом универсуме возвращается пустое мультимножество.

Листинг 6: Функция `generateAutomaticMultiset`

```
MultiSet generateAutomaticMultiset(const MultiSet &universe,
                                   std::size_t power)
{
    MultiSet result;
    if (power == 0 || universe.empty()) return result;
    std::vector<std::string> pool;
    pool.reserve(universe.size());
    for (const auto &[value, count] : universe.data())
        for (std::size_t i = 0; i < count; ++i)
```

```
            pool.push_back(value);
    std::random_device rd;
    std::mt19937_64 generator(rd());
    const std::size_t actualPower = std::min(power, pool.size());
    std::shuffle(pool.begin(), pool.end(), generator);
    for (std::size_t i = 0; i < actualPower; ++i)
        result.add(pool[i]);
    return result;
}
```

2.2.6 printUniverse

Вход: universe (const MultiSet&) – универсум; order (const std::vector<std::string>&) – порядок элементов универсума.

Выход: отсутствует (результат выводится в стандартный поток вывода).

Описание: функция выводит число уникальных элементов универсума и его общую мощность, а затем построчно печатает каждый элемент с порядковым номером и кратностью. Нумерация соответствует той, которая используется при ручном заполнении мультимножеств. Для пустого универсума выводится символ ∅.

Листинг 7: Функция `printUniverse`

```
void printUniverse(const MultiSet &universe,
                   const std::vector<std::string> &order)
{
    std::cout << "\nУниверсум (" << order.size()
              << " уникальных элементов, общая мощность "
              << universe.size() << "):\n";
    if (order.empty()){
        std::cout << "∅\n";
        return;
    }
    std::size_t i = 1;
```

```
    for (const auto &value : order){
        std::cout << i++ << ": " << value
                  << " (кратность " << universe.count(value) << ")\n";
    }
}
```

2.2.7 printMultiset

Вход: name (const std::string&) – имя мультимножества; set (const MultiSet&) – выводимое мультимножество.

Выход: отсутствует (результат выводится в стандартный поток вывода).

Описание: функция печатает мультимножество в виде набора пар «элемент (кратность)» в фигурных скобках, а после него – мощность мультимножества. Пустое мультимножество выводится как ∅.

Листинг 8: Функция `printMultiset`

```
void printMultiset(const std::string &name, const MultiSet &set)
{
    std::cout << name << " = ";
    if (set.empty()) std::cout << "∅";
    else{
        std::cout << "{ ";
        bool first = true;
        for (const auto &[value, count] : set.data()){
            if (!first) std::cout << ", ";
            std::cout << value << " (" << count << ")";
            first = false;
        }
        std::cout << " }";
    }
    std::cout << "  |" << name << "| = " << set.size() << '\n';
}
```

2.2.8 setUnion

Вход: a (const MultiSet&) – первое мультимножество; b (const MultiSet&) – второе мультимножество.

Выход: мультимножество (MultiSet) – объединение $A \cup B$.

Описание: результат инициализируется копией мультимножества A. Затем для каждого элемента B проверяется, превышает ли его кратность кратность в результате, и если да, добавляется недостающая разность. Таким образом, кратность каждого элемента в объединении равна максимуму из его кратностей в A и B.

Листинг 9: Функция `setUnion`

```
MultiSet setUnion(const MultiSet &a, const MultiSet &b)
{
    MultiSet result = a;
    for (const auto &[value, count] : b.data())
        result.add(value, count > result.count(value) ? count - result.count(
   value) : 0);
    return result;
}
```

2.2.9 intersection

Вход: a (const MultiSet&) – первое мультимножество; b (const MultiSet&) – второе мультимножество.

Выход: мультимножество (MultiSet) – пересечение $A \cap B$.

Описание: функция обходит элементы A и для каждого определяет кратность в B. Кратность элемента в пересечении равна минимуму из его кратностей

в A и B. Элементы с нулевой итоговой кратностью в результат не попадают.

Листинг 10: Функция `intersection`

```
MultiSet intersection(const MultiSet &a, const MultiSet &b)
{
    MultiSet result;
    for (const auto &[value, countA] : a.data()){
        const auto countB = b.count(value);
        const auto count = std::min(countA, countB);
        if (count > 0) result.add(value, count);
    }
    return result;
}
```

2.2.10 complement

Вход: a (const MultiSet&) – исходное мультимножество; universe (const MultiSet&) – универсум.

Выход: мультимножество (MultiSet) – дополнение $\overline{A}$ до универсума.

Описание: функция обходит элементы универсума и для каждого вычисляет разность кратности в универсуме и кратности в A. Если разность положительна, элемент добавляется в результат с этой кратностью.

Листинг 11: Функция `complement`

```
MultiSet complement(const MultiSet &a, const MultiSet &universe)
{
    MultiSet result;
    for (const auto &[value, countU] : universe.data()){
        const auto countA = a.count(value);
        if (countU > countA)
            result.add(value, countU - countA);
    }
    return result;
}
```

2.2.11 difference

Вход: a (const MultiSet&) – уменьшаемое; b (const MultiSet&) – вычитаемое; universe (const MultiSet&) – универсум.

Выход: мультимножество (MultiSet) – разность $A \setminus B$.

Описание: функция выражает разность через уже реализованные операции: она находит пересечение A с дополнением B до универсума, то есть $A \setminus B = A \cap \overline{B}$.

Листинг 12: Функция `difference`

```
MultiSet difference(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe)
{
    return intersection(a, complement(b, universe));
}
```

2.2.12 symmetricDifference

Вход: a (const MultiSet&) – первое мультимножество; b (const MultiSet&) – второе мультимножество; universe (const MultiSet&) – универсум.

Выход: мультимножество (MultiSet) – симметрическая разность $A \triangle B$.

Описание: функция вычисляет симметрическую разность как разность объединения и пересечения мультимножеств: $A \triangle B = (A \cup B) \setminus (A \cap B)$. Для этого используются функции `setUnion`, `intersection` и `difference`.

Листинг 13: Функция `symmetricDifference`

```
MultiSet symmetricDifference(const MultiSet &a, const MultiSet &b, const
    MultiSet &universe)
```

```
{
    return difference(setUnion(a, b), intersection(a, b), universe);
}
```

2.2.13 arithmeticSum

Вход: a (const MultiSet&) – первое слагаемое; b (const MultiSet&) – второе слагаемое; universe (const MultiSet&) – универсум.

Выход: мультимножество (MultiSet) – арифметическая сумма $A + B$.

Описание: функция вычисляет арифметическую сумму мультимножеств, при которой кратности одинаковых элементов складываются, но не могут превышать кратность этого элемента в универсуме. Результат инициализируется копией A, затем для элементов B вычисляется добавляемая кратность, ограниченная кратностью в универсуме.

Листинг 14: Функция `arithmeticSum`

```
MultiSet arithmeticSum(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe)
{
    MultiSet result = a;
    for (const auto &[value, countA] : b.data()){
        const auto count = std::min(universe.count(value), countA + b.count(
    value));
        result.add(value, count);
    }
    return result;
}
```

2.2.14 arithmeticDifference

Вход: a (const MultiSet&) – уменьшаемое; b (const MultiSet&) – вычитаемое.

Выход: мультимножество (MultiSet) – арифметическая разность $A - B$.

Описание: для каждого элемента A из его кратности вычитается кратность в B. Элемент попадает в результат только если разность положительна, то есть отрицательные и нулевые кратности отбрасываются.

Листинг 15: Функция `arithmeticDifference`

```
MultiSet arithmeticDifference(const MultiSet &a, const MultiSet &b)
{
    MultiSet result;
    for (const auto &[value, countA] : a.data()){
        const auto countB = b.count(value);
        if (countA > countB)
            result.add(value, countA - countB);
    }
    return result;
}
```

2.2.15 arithmeticProduct

Вход: a (const MultiSet&) – первый множитель; b (const MultiSet&) – второй множитель; universe (const MultiSet&) – универсум.

Выход: мультимножество (MultiSet) – арифметическое произведение $A \cdot B$.

Описание: функция обходит элементы A, которые присутствуют и в B. Кратность элемента в результате равна произведению его кратностей в A и B, ограниченному кратностью этого элемента в универсуме. Элементы, отсутствующие в B, в результат не попадают.

Листинг 16: Функция `arithmeticProduct`

```
MultiSet arithmeticProduct(const MultiSet &a, const MultiSet &b, const
   MultiSet &universe)
```

```
{
    MultiSet result;
     for (const auto &[value, countA] : a.data()){
         const auto countB = b.count(value);
         if (countB > 0){
             const auto count = std::min(countA * countB, universe.count(value
    ));
             result.add(value, count);
         }
     }
     return result;
}
```

2.2.16 arithmeticDivision

Вход: a (const MultiSet&) – делимое; b (const MultiSet&) – делитель.

Выход: мультимножество (MultiSet) – арифметическое частное A/B.

Описание: для каждого элемента A, который присутствует в B и кратность которого в A не меньше кратности в B, в результат добавляется элемент с кратностью, равной целой части от деления кратности в A на кратность в B. Остальные элементы в результат не включаются.

Листинг 17: Функция `arithmeticDivision`

```
MultiSet arithmeticDivision(const MultiSet &a, const MultiSet &b)
{
    MultiSet result;
    for (const auto &[value, countA] : a.data()){
        const auto countB = b.count(value);
        if (countB > 0 && countA >= countB)
            result.add(value, countA / countB);
    }
    return result;
}
```

2.2.17 printOperations

Вход: a (const MultiSet&) – мультимножество A; b (const MultiSet&) – мультимножество B; universe (const MultiSet&) – универсум.

Выход: отсутствует (результаты выводятся в стандартный поток вывода).

Описание: функция последовательно вычисляет и выводит результаты всех операций над мультимножествами: объединение, пересечение, разности, симметрическую разность и дополнения, а затем арифметические операции (сумму, разности, произведение и деление). Вызовы арифметических операций обёрнуты в блоки `try-catch`, поэтому при возникновении исключения выводится сообщение об ошибке, а выполнение продолжается со следующей операции.

Листинг 18: Функция `printOperations`

```
void printOperations(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe)
{
    std::cout << "\nОПЕРАЦИИ НАД МУЛЬТИМНОЖЕСТВАМИ:\n\n";
    printMultiset("A ∪ B", setUnion(a, b));
    printMultiset("A ∩ B", intersection(a, b));
    printMultiset("A \\ B", difference(a, b, universe));
    printMultiset("B \\ A", difference(b, a, universe));
    printMultiset("A △ B", symmetricDifference(a, b, universe));
    printMultiset("Ā", complement(a, universe));
    printMultiset("B̄", complement(b, universe));
    printMultiset("\nA + B", arithmeticSum(a, b, universe));
    printMultiset("A - B", arithmeticDifference(a, b));
    printMultiset("B - A", arithmeticDifference(b, a));
    printMultiset("A * B", arithmeticProduct(a, b, universe));
    printMultiset("A / B", arithmeticDivision(a, b));
    printMultiset("B / A", arithmeticDivision(b, a));
}
```

2.2.18 createMultisets

Вход: bits (std::size_t&) – разрядность кода Грея; universe (MultiSet&) – универсум; universeOrder (std::vector<std::string>&) – порядок элементов универсума; a (MultiSet&) – мультимножество A; b (MultiSet&) – мультимножество B.

Выход: значение типа bool: true при успешном создании мультимножеств, false при ошибке генерации универсума.

Описание: функция генерирует коды Грея заданной разрядности и формирует из них универсум, присваивая каждому элементу случайную кратность от 1 до 7. Затем выводится универсум и время выполнения этапов генерации, заполнения и печати. После этого пользователь вводит мощности мультимножеств A и B и выбирает способ их заполнения (вручную или автоматически), и мультимножества создаются соответствующими функциями. При нулевой разрядности универсум и оба мультимножества остаются пустыми.

Листинг 19: Функция `createMultisets`

```
bool createMultisets(std::size_t &bits, MultiSet &universe, std::vector<std::
    string> &universeOrder, MultiSet &a, MultiSet &b)
{
    const auto t0 = std::chrono::steady_clock::now();
    std::vector<std::string> codes;
    codes = generateGrayCode(bits);
    universe.clear();
    universeOrder.clear();
    if (bits == 0){
        a.clear();
        b.clear();
        std::cout << "\nРазрядность равна 0.\n";
        printMultiset("U", universe);
        printMultiset("A", a);
        printMultiset("B", b);
```

```
15          return true;
16      }
17      std::random_device rd;
18      std::mt19937_64 generator(rd());
19      std::uniform_int_distribution<std::size_t> dist(1, 7);
20      for (const auto &code : codes){
21          universe.add(code, dist(generator));
22          universeOrder.push_back(code);
23      }
24      printUniverse(universe, universeOrder);
25      std::cout << "generateGrayCode: " << genMs << " ms\n";
26      std::cout << "заполнение MultiSet: " << fillMs << " ms\n";
27      std::cout << "пачать MultiSet: " << printMs << " ms\n";
28      const std::size_t maxPower = universe.size();
29      const std::size_t powerA = readInt("\nВведите мощность мультимножества A:
        ", 0, maxPower);
30      const std::size_t powerB = readInt("Введите мощность мультимножества B: "
        , 0, maxPower);
31      const int modeA = readInt("\nСпособ заполнения A (1 - вручную, 2 - автома
        тически): ", 1, 2);
32      const int modeB = readInt("Способ заполнения B (1 - вручную, 2 - автомати
        чески): ", 1, 2);
33      if (modeA == 1)
34          a = readManualMultiset("A", universe, universeOrder, powerA);
35      else
36          a = generateAutomaticMultiset(universe, powerA);
37      if (modeB == 1)
38          b = readManualMultiset("B", universe, universeOrder, powerB);
39      else
40          b = generateAutomaticMultiset(universe, powerB);
41      std::cout << "\nСОЗДАННЫЕ МНОЖЕСТВА:\n";
42      printMultiset("A", a);
43      printMultiset("B", b);
44      return true;
45  }
```

2.2.19 menu

Вход: отсутствует.

Выход: отсутствует.

Описание: функция реализует главное меню программы и хранит текущие универсум и мультимножества A и B. В цикле пользователю предлагается создать новые мультимножества (с вводом разрядности не более 20), вывести универсум и мультимножества, выполнить операции над ними или выйти из программы. Выбранный пункт обрабатывается оператором `switch`, цикл завершается при выборе пункта «Выход».

Листинг 20: Функция `menu`

```
void menu()
{
    constexpr std::size_t MAX_BITS = 20;
    std::size_t bits = 0;
    MultiSet universe;
    std::vector<std::string> universeOrder;
    MultiSet a;
    MultiSet b;
    while (true){
        std::cout << "\n1. Создать новые мультимножества\n";
        std::cout << "2. Показать универсум и мультимножества\n";
        std::cout << "3. Выполнить операции над мультимножествами\n";
        std::cout << "0. Выход\n";
        const int choice = readInt("Выберите пункт: ", 0, 3);
        switch (choice){
        case 1:
            bits = readInt("\nВведите разрядность бинарного кода Грея: ", 0,
   MAX_BITS);
            createMultisets(bits, universe, universeOrder, a, b);
            break;
        case 2:
            std::cout << '\n';
```

```
22              printUniverse(universe, universeOrder);
23              printMultiset("A", a);
24              printMultiset("B", b);
25              break;
26          case 3:
27              printMultiset("A", a);
28              printMultiset("B", b);
29              printOperations(a, b, universe);
30              break;
31          case 0:
32              return;
33          }
34      }
35  }
```

2.2.20 main

Вход: отсутствует.

Выход: код завершения программы (int), 0 при нормальном завершении.

Описание: точка входа в программу. Функция запускает главное меню и после его завершения возвращает ноль.

Листинг 21: Функция `main`

```
1  int main(){
2      menu();
3      return 0;
4  }
```

2.3 Алгоритм автоматического заполнения

Автоматическое заполнение реализовано функцией `generateAutomaticMultiset` и представляет собой случайную выборку без возвращения из универсума. Пусть

требуется получить мультимножество мощности p. Алгоритм выполняется в три шага:

1. универсум разворачивается в линейный список («пул»), в котором каждый элемент повторяется столько раз, какова его кратность в U, поэтому размер пула равен $|U|$;
2. пул перемешивается функцией `std::shuffle` с генератором `std::mt19937_64`, который инициализируется значением `std::random_device`;
3. первые p значений перемешанного пула по одному добавляются в результат методом `add`.

Корректность алгоритма следует из его устройства. Значение не может быть извлечено из пула чаще, чем оно в нём встречается, поэтому для любого x выполняется $m_A(x) \leqslant m_U(x)$ и дополнительных проверок не требуется. Из пула берётся ровно p значений, значит $|A| = p$.

Время работы и дополнительная память составляют $O(|U|)$, к этому добавляется $O(p \log |X|)$ на вставки в `std::map`.

2.4 Ручное заполнение мультимножеств

При ручном заполнении пользователь выбирает элемент по номеру из списка, который выводит функция `printUniverse`. Нумерация соответствует порядку кодов Грея, сохранённому в векторе `universeOrder`. Для каждого из p вхождений выполняются следующие действия:

1. номер запрашивается через `readInt` в диапазоне от 1 до числа уникальных элементов универсума;
2. по номеру из вектора `universeOrder` выбирается код `chosen`;

3. если кратность `chosen` в результате уже равна его кратности в универсуме, выводится сообщение об ошибке, счётчик вхождений уменьшается (`-i`) и запрос повторяется для того же вхождения;

4. иначе элемент добавляется в результат с кратностью 1.

Таким образом, мультимножество всегда остаётся подмультимножеством универсума, а итоговая мощность в точности равна введённой.

2.5 Защита от некорректного ввода

Все числовые значения (пункт меню, разрядность, мощности, способ заполнения, номер элемента) вводятся функцией `readInt`. Строка считывается целиком функцией `std::getline` и разбирается через `std::stringstream`: после целого числа не должно остаться ни одного непробельного символа. Это исключает частичное чтение, свойственное оператору `cin >> x`, при котором строка «12abc» была бы воспринята как 12. Реакция программы на характерные случаи приведена в таблице 2.

Таблица 2 – Реакция программы на некорректный ввод

Ввод	**Вывод**
`abc`, `12abc`, `1.5`, `3 7`, пустая строка	сообщение об ошибке с допустимым диапазоном, повторный запрос
число вне диапазона (разрядность 21 или -1, мощность больше $\lvert U\rvert$, номер пункта меню 4)	сообщение об ошибке с допустимым диапазоном, повторный запрос
число, не помещающееся в `long long`	поток переходит в состояние ошибки, значение отклоняется, аварийного завершения нет
число с пробелами по краям, например « `5` »	принимается как 5
номер элемента вне диапазона $[1, N]$ при ручном вводе	сообщение об ошибке, повторный запрос
элемент, исчерпавший свою кратность в универсуме	сообщение с указанием максимальной кратности, ввод для этого вхождения повторяется
действия над пустыми A и B (универсум ещё не создан)	программа не завершается аварийно, пустые мультимножества выводятся как $\varnothing$
деление на элемент с нулевой кратностью	элемент пропускается, деление на ноль не выполняется

3 Результаты работы

Для проверки программа запускалась в консоли. Ниже приведены сценарии корректного и некорректного ввода. Для каждого сценария указаны вводимые значения и ожидаемое поведение программы.

Корректный ввод

3.1 Главное меню

После запуска программа выводит меню из четырёх пунктов: создание новых мультимножеств, показ универсума и мультимножеств, выполнение операций и выход (рис. **??**).

3.2 Создание универсума

Выбирается пункт 1 и вводится разрядность 3. Программа формирует 8 кодов Грея, присваивает каждому случайную кратность от 1 до 7 и выводит нумерованный список универсума с его мощностью. Затем печатаются затраты времени на генерацию кодов, заполнение и печать универсума (рис. **??**). Коды в списке идут в порядке кода Грея (таблица 1), поэтому соседние строки отличаются одним разрядом.

3.3 Автоматическое заполнение

После создания универсума запрашиваются мощности A и B. Вводится мощность A, равная 5, и способ заполнения 2 (автоматически). Результат (рис. **??**) должен содержать ровно 5 вхождений, а кратность каждого элемента не должна превышать его кратность в универсуме.

3.4 Ручное заполнение

Для мультимножества B задаётся мощность 4 и способ заполнения 1 (вручную). Вводятся номера элементов универсума 1, 3, 2, 4. Программа запрашивает номер для каждого вхождения, поэтому в итоговое мультимножество должны попасть четыре различных элемента с кратностью 1 (рис. **??**).

3.5 Выполнение операций

Далее выбирается пункт 3 меню, и программа выводит результаты всех операций для созданных мультимножеств (рис. **??**). Они сверяются с формулами таблицы **??**: кратность каждого элемента результата лежит в пределах от 0 до $m_U(x)$, а мощности совпадают с суммой кратностей.

3.6 Граничные случаи

Проверяются три граничных случая: разрядность 0, мощность мультимножества 0 и мощность, равная $|U|$.

При разрядности 0 универсум пуст, поэтому программа сразу сообщает о нулевой разрядности и выводит $U = \varnothing$, $A = \varnothing$ и $B = \varnothing$ без запроса мощностей (рис. **??**).

Затем создаётся универсум ненулевой разрядности, для A задаётся мощность 0, а для B – мощность $|U|$. В этом случае $A = \varnothing$, а B совпадает с универсумом (рис. **??**).

3.7 Время выполнения этапов

Функция `createMultisets` измеряет время генерации кодов Грея, заполнения универсума и его печати. Замеры для нескольких разрядностей приведены в таблице 3.

Таблица 3 – Время выполнения этапов, мс

n	2^n	Генерация кодов	Заполнение `MultiSet`	Печать
5	32	...	...	...
10	1024	...	...	...
15	32768	...	...	...
20	1048576	...	...	...

Генерация имеет сложность $O(n \cdot 2^n)$, заполнение – $O(2^n \log 2^n)$ из-за вставок в упорядоченный словарь. При увеличении n на единицу число элементов удваивается, поэтому время каждого этапа должно расти примерно вдвое.

Некорректный ввод

3.8 Некорректный пункт меню

В меню вводятся значения `abc`, `4`, `-1` и пустая строка. Каждое значение отклоняется сообщением с допустимым диапазоном, после чего меню запрашивает пункт повторно (рис. **??**).

3.9 Некорректная разрядность

При создании универсума вводятся значения `21`, `-1`, `3.5` и `5abc`. Первые два значения лежат вне диапазона от 0 до 20, остальные содержат посторонние символы. Все четыре значения должны быть отклонены (рис. **??**).

3.10 Мощность больше мощности универсума

При запросе мощности мультимножества вводится число, на единицу большее $|U|$. Программа сообщает допустимый диапазон $[0, |U|]$ и повторяет запрос (рис. **??**).

3.11 Ошибки при ручном вводе

При ручном заполнении вводятся номер 0, номер, на единицу больший числа элементов универсума, и нечисловая строка. Затем при мощности $p = |U|$ один и тот же номер вводится подряд, пока его кратность не исчерпана. Для каждой ошибки программа выводит сообщение, причём при исчерпании кратности называет её максимальное значение. Введённое вхождение при этом не засчитывается (рис. **??**).

3.12 Операции над пустыми мультимножествами

Пункты 2 и 3 выбираются сразу после запуска, до создания универсума. Программа не завершается аварийно: универсум и оба мультимножества выводятся как $\varnothing$, и все операции дают пустой результат (рис. **??**).

Заключение

В ходе лабораторной работы изучены мультимножества, операции над ними и алгоритм построения бинарного кода Грея. На языке C++ разработана программа, которая:

- генерирует бинарный код Грея разрядности $n \in [0, 20]$ по формуле $G = BIN \oplus (BIN \gg 1)$;
- строит универсум из 2^n кодов со случайными кратностями от 1 до 7;
- формирует мультимножества A и B заданной мощности вручную (выбором номера элемента) или автоматически (случайной выборкой без возвращения);
- вычисляет и выводит объединение, пересечение, обе разности, симметрическую разность, оба дополнения, а также арифметические сумму, разности, произведение и частное;
- проверяет ввод пользователя, управляется через меню и выводит время выполнения этапов построения универсума.

Достоинства:

- хранение вынесено в класс `MultiSet`, поэтому операции работают через его интерфейс и не зависят от внутреннего представления;
- словарь `std::map` хранит только элементы с ненулевой кратностью, поэтому результаты операций не содержат нулевых вхождений;
- разность и симметрическая разность выражены через базовые операции, а код совпадает с математическими определениями;
- при ручном вводе выбирается номер элемента, а не строка, поэтому ошиб-

ки в длине и символах кода невозможны;

- автоматическое заполнение гарантированно остаётся внутри универсума.

Недостатки:

- доступ к кратности через `std::map` требует $O(\log |X|)$ сравнений строк длины n, тогда как массив кратностей по номеру кода работал бы за $O(1)$;
- автоматическое заполнение строит пул из $|U|$ строк; при $n = 20$ это миллионы строк и значительный расход памяти;
- при ручном вводе каждое вхождение вводится отдельно, поэтому заполнение мультимножества большой мощности неудобно;
- при $n = 20$ вывод универсума занимает около миллиона строк, что на практике делает работу с максимальной разрядностью неудобной.

Масштабирование. Программу можно расширить, не меняя уже написанные функции:

- заменить `std::map<std::string, Count>` вектором кратностей, индексированным номером кода, чтобы ускорить все операции;
- перегрузить операторы `+`, `-`, `*`, `/` и `|`, `&` для класса `MultiSet`, чтобы выражения над мультимножествами записывались в естественной форме;
- добавить проверку включения $A \subseteq B$ и равенства мультимножеств по кратностям;
- добавить в меню пункт, сохраняющий результат операции в A или B, что позволит строить цепочки вычислений;
- реализовать чтение и запись мультимножеств из файла.

Список литературы

1. Новиков Ф. А. Дискретная математика для программистов: учебник для вузов. – 3-е изд. – СПб.: Питер, 2009. – 384 с.

2. Дискретная математика: код Грея [Электронный ресурс]. – URL: https://istina.msu.r (дата обращения: ДД.ММ.2026).

3. Алгоритмы построения кодов Грея [Электронный ресурс]. – URL: https://www.autex (дата обращения: ДД.ММ.2026).

4. cppreference.com: справочник по языку C++ и стандартной библиотеке [Электронный ресурс]. – URL: https://en.cppreference.com (дата обращения: ДД.ММ.2026).

Приложение. Исходный код программы